

[Open in app](#)

Medium

 Search

★ Member-only story

Why You (Currently) Do Not Need Deep Learning for Time Series Forecasting

What you need instead: Learnings from the Makridakis M5 competitions and the 2023 Kaggle AI report



Jonte Dancker · Follow

Published in Towards Data Science

11 min read · Jun 20, 2024

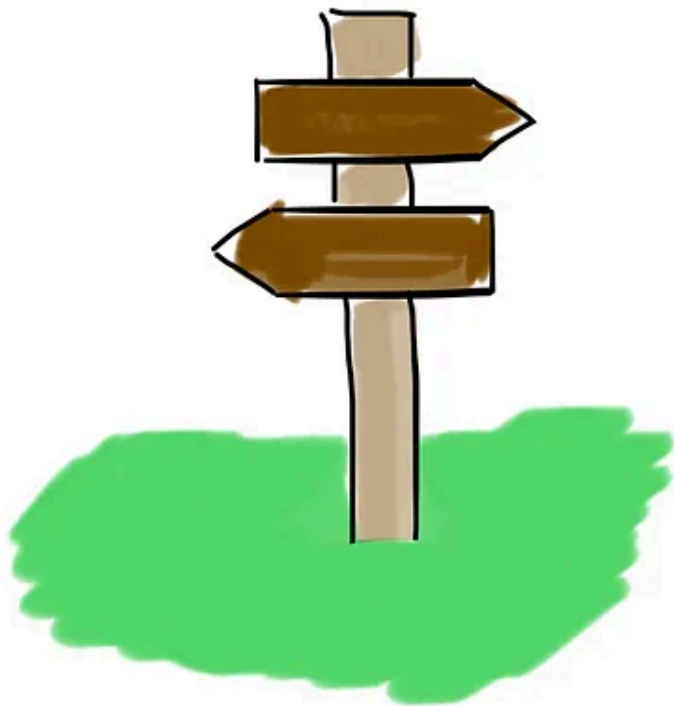
 Listen Share More

Image by the author.

Deep Learning for Time Series Forecasting receives a lot of attention. Many articles and scientific papers write about the latest Deep Learning model and how it is much better than any ML or statistical model. This gives the impression that Deep

Learning will solve all our problems for time series forecasting. In particular for new people in the field.

But from my experience, Deep Learning is not what you need. Other things are more important and work better for time series forecasting.

Hence, in this article, I want to show you what works. I will show you things that have proven themselves in many ways. I will use the findings of the Makridakis M5 competitions and the Kaggle AI Report 2023 and compare them against my experience.

The Makridakis competitions compare forecasting methods on real-world data sets. They show what works in practice. Since the start of the competitions almost 40 years ago, their findings have changed. In the first three competitions, M1 to M3, statistical models dominated the field. In the M4 competition, ML models began to show their potential in the form of hybrid approaches, which combine ML models and statistical methods.

In the M5 competition, participants had to predict the hierarchical unit sales of Walmart. The competition was split into an Accuracy and Uncertainty competition. The goal of the Accuracy competition was to find the best point forecast for each time series. The Uncertainty competition focused on probabilistic forecasts. For this, participants had to predict nine different quantiles describing the complete distribution of future sales.

The Kaggle AI Report, in contrast, contains a collection of essays written by the Kaggle community as part of a Kaggle competition. Usually, they collect key learnings and trends from recent high-performing solutions.

As a side note, Kaggle hosted the M5 competition. Thus, the competition attracted many people who participated in other Kaggle competitions. This in turn might be the reason why the results of the Kaggle AI Report and the M5 competitions are very similar.

But let's see what the findings are.

ML models show superior performance

Over the past years, ML approaches have taken over the field of time series forecasting. In the M4 competition from 2020, they started to become important as part of hybrid approaches. But two years later in the M5 competitions ML dominated the competition. All top-performing solutions were pure ML approaches, beating all statistical benchmarks.

Particularly Gradient Boosting Machines (GBMs) dominate both M5 and Kaggle competitions. The most successful ones are LightGBM, XGBoost, and CatBoost. Their effective handling of many features, little to no data pre-processing and transformation, fast training, and ability to quantify feature importance make them one of the best off-the-shelf models in recent years.

They are very convenient for experimenting and allow a fast iteration, which is important in identifying the best features. Moreover, we only need to optimize a few hyperparameters. Often the default settings already result in good performance. Hence, they have become the go-to algorithm for many time series problems.

Moreover, these approaches beat deep learning models in performance and training time. Hence, Deep Learning models are less popular.

My first choice are GBMs as well due to the above-stated advantages. I moved from XGBoost over CatBoost to LightGBM over time. In many problems, LightGBM had a better performance and shorter training time. Also, LightGBM was more capable for smaller data sets. For example, for one problem I trained a CatBoost model and a LightGBM model using the same features and loss function. The CatBoost took roughly 50% more time to train and resulted in an MAE of 2% greater than the LightGBM.

Statistical methods are still valuable

Although ML methods often outperform statistical models, we should not forget statistical models. 92.5 % of the teams in the M5 accuracy competition could not beat the simple baseline method which was an off-the-shelf forecasting method. Using an ML model does not guarantee the best performance. Moreover, they take more time to develop.

Thus, we should always start with a simple model before starting on any ML model. We can use them as baseline models to support our decision-making. For example, does the ML model add enough value to balance the added complexity?

If you want to read more about baseline models and why you should start with them, check out my article.

Why You Should Always Start With a Baseline Model

A baseline model takes 10 % of the time to develop but gets us 90 % of the way to achieve reasonable results.

pub.towardsai.net

I usually start with the simplest possible model as my baseline. From my experience, a simple statistical baseline model is often hard to beat. Moreover, it only takes a small fraction of the time needed compared to developing an ML model.

Ensembles improve performance

The M5 and Kaggle competitions show that combining different models often results in better forecast accuracy. Ensembling is particularly successful when the individual models make uncorrelated errors. For example, some teams in the M5 competitions combined models trained on subsets of the training data and with different loss functions.

We can build ensembles through simple averaging (blending), complex blending, or stacking approaches. However, often an equal weighting is enough.

Although ensembling improves model performance, they have drawbacks in real-world applications. They add complexity which reduces explainability and is harder to maintain in a production environment. Hence very complex ensembles are often not used. A simpler alternative usually provides good enough results while being easier to maintain.

I have seen the benefit of using ensembles in real-world applications. For me, two approaches usually worked best. One approach is to combine models of the same type, like LightGBM, which are trained on different loss functions. The other approach is to use models which are trained with different features. I usually go

with these two approaches as they are easy to implement and fast to test. I only need to change the loss function or the model input but can leave the rest of my pipeline exactly the same.

Moreover, I usually focus on simple ensembles to keep the complexity as small as possible. Usually, I use a simple averaging. However, the benefit of the ensemble must be large enough to make it into production. And usually, the ensemble does not improve the performance enough.

Scientific literature had a small effect on applied time series forecasting

We see a gap between the scientific literature and applied ML forecasting for time series. The scientific literature mostly focuses on Deep Learning models. These, however, are not used in practice.

But how can it be that there is such a large gap? Papers show how their models beat ML and statistical models. Why is nobody using them then? Because Deep Learning approaches are often not successful in real-world applications. Moreover, they can be very costly to train.

But why is the scientific literature focused so much on Deep Learning if it is not practical?

Well, I can only guess, but Deep Learning catches more attention than other ML models. I have seen this with both my articles about N-BEATS and N-HiTS. Everybody wants to work on Deep Learning since the great success of LLMs for NLP.

N-BEATS — The First Interpretable Deep Learning Model That Worked for Time Series Forecasting

An easy-to-understand deep dive into how N-BEATS works and how you can use it.

towardsdatascience.com

N-HiTS — Making Deep Learning for Time Series Forecasting More Efficient

A deep dive into how N-HiTS works and how you can use it

towardsdatascience.com

Deep Learning approaches like Transformers work well for NLP but not for time series. Although both tasks might look similar as both are sequences of values, they are not. In NLP the context matters, while for time series the order matters. This is a small difference that has a big impact on the approaches that work well.

Hence, you might ask yourself, if it is worth training a Deep Learning model, which probably will perform worse than an ML model. I have tried some approaches like N-BEATS and N-HiTS. But these models were always performing worse and took a lot longer to train than my ML models. Hence, I have never successfully applied them in a real-world forecasting task.

Now I have talked a lot about what models work and which do not. But focusing only on the models is not enough. Other things are even more important than choosing the right model. The Kaggle AI Report and the M5 competition conclude that good feature engineering is more important than the model. It is probably the most important aspect when developing a new model.

Feature engineering is more important than models

Real-world data is messy and needs a lot of cleaning to be of use. Hence, we need a lot of time to clean and understand the data to find good features. This is where I spent most of my time when developing a model.

In Kaggle competitions, feature engineering has been repeatedly proven to be crucial. Often the quality of the features makes the difference between solutions. The team that can extract the most information from the data and create a greater separation between features often had the better-performing model. Hence, spending time creating, selecting, and transforming features is crucial.

But what is a good approach to feature engineering?

From my experience, there is no single best approach. It is rather a trial-and-error approach to trying different features and feature combinations. What has helped me to find good features is being creative and flexible and using a data-driven approach with a good cross-validation strategy.

Sometimes it helps to take a broad approach, generating many features. For example, if I have many exogenous variables and external data sources available, I usually start with using them without any transformation. Here, the choice of variables depends on the results of my explanatory data analysis and what I find there.

Sometimes, it is better to concentrate on a single feature and expand it in several ways. For example, if you have a limited amount of data. How I expand a single feature depends on the problem I want to solve. Sometimes using window features such as the mean, standard deviation, or min and max results in a performance boost. Sometimes it is enough to use different lag features.

The M5 and Kaggle competitions show that domain knowledge is not needed to find good features. However, domain knowledge has often helped me to understand the data faster and derive better features.

Exogenous/explanatory variables can boost performance

Using external data is critical for improving the forecasting performance. They can give a strong boost to the model's performance. In the M5 competitions, models that used external data performed better than models that only relied on historical data. Hence, finding these explanatory variables is crucial in real-world applications.

I try to identify as many external factors as possible. Then I test these during my feature engineering process. These can be simple date-time-related features, such as holidays, or data from other sources. My choice depends on the data availability and accessibility during inference.

In practice, I saw the greatest boost of performance happen when I added exogenous variables. Because in most real-world applications, the behavior of the time series you want to forecast depends on external factors. For example, if you want to forecast electricity prices, using the electricity consumption and generation as features gives a great performance boost. Why? Because electricity prices usually increase when electricity consumption increases or the electricity generation decreases.

Iterate as fast as possible

As we usually try many features in our model development process, we must be as fast as possible. We do not want to wait long to see if adding a new feature is helpful.

For example, the winning solution in the M5 accuracy competition tested 220 models.

Hence, Kagglers often use LightGBM as their go-to model as the model trains very fast. Thus, they can run many experiments in a short period.

Again, I agree with the findings. I usually get better results the more things I can test. More features, different feature combinations and different loss functions. Being fast helps me to test many hypotheses in a shorter time.

However, to decide which features improve performance, we must repeatedly evaluate how good our model is. We need an approach that we can trust and that helps us identify the best features and models.

Effective cross-validation strategies are crucial

An effective cross-validation strategy is critical to choose the best model objectively. Building a local validation process using cross-validation helps us...

- understand if the model performance is reliably improved
- uncover areas in which the model is making mistakes or unreliable predictions
- guide our feature engineering
- simulate post-sample accuracy
- avoid overfitting on test data
- mitigate uncertainty
- tune the hyperparameters of the model

The M5 and Kaggle competitions show this importance. Many teams in the M5 competition failed to choose the best model for their final submission. In Kaggle competitions, the best models in the public leaderboard are often not the competition winners.

However, choosing a good cross-validation strategy can be difficult, there are many options.

- What period do we select?
- What is the size of the validation windows?
- How do we update the windows?
- What criteria/metrics do we use to summarize the forecasting performance?

Hence, the strategy should always be problem-dependent.

Every problem needs a unique approach

The results of the Kaggle and M5 competitions show that we need a unique approach for every data set and problem. There is no off-the-shelf solution.

To decide which approach works, we need our knowledge and experience.

We must adjust the models based on the intricacies of the forecasting task. This includes all the points I have discussed above. From the feature engineering to the cross-validation to the choice of model.

For example, in the M5 accuracy competition, combining the best models for each time series beat the winning solution by roughly two percent.

This is why deep learning models currently do not work in real-world applications. They try to be a one-size-fits-all solution to a problem that needs a custom solution. Deep learning models promise to make our lives easier by taking away the need for detailed feature engineering. For example, N-BEATS and N-HiTS promise to automatically find seasonality and trend components in the time series. However, they do not find the unique intricacies of a time series. In contrast, with our knowledge we can find these intricacies and encode them in features that ML models can use and thus beat Deep Learning models.

Conclusion

The M5 competitions and the 2023 Kaggle AI report agree on what works and is important for time series forecasting. I can only support their conclusion from my experience working on applied time series forecasting tasks.

The most important factors are:

- ML models show superior performance
- Statistical methods are still valuable
- Ensembles improve performance
- Scientific literature had a small effect on applied time series forecasting
- Feature engineering is more important than models
- Exogenous/explanatory variables can boost performance
- Iterate as fast as possible
- Effective cross-validation strategies are crucial
- Every problem needs a unique approach

As you can see Deep Learning is not part of the list.

Deep Learning currently is not good enough. These models have not successfully moved from the literature to real-world problems. However, this might change in the future. We saw a shift from statistical models, which dominated time series forecasting for a long time, to ML models. Such a shift might happen in the future to Deep Learning models. Hence, it is good to stay up-to-date on the development of Deep Learning. But currently, they are not useful in real-world applications today.

Hence, there is still a tremendous potential for further research and improvement, not only on the Deep Learning side but also on the ML side.

I hope that you find a lot of good stuff in this article. I hope that it will help you to improve the performance of your next time series forecasting models.

See you in my next article and/or leave a comment.

[Machine Learning](#)[Time Series Forecasting](#)[Deep Learning](#)[Time Series Analysis](#)[Tips And Tricks](#)